Series and TimeSeries

Series is a list of values of same types. The array in nadi can be different types, so this is a separate data type from that.

In addition to arrays, Series values can be null, while arrays or attributes can not be null.

TimeSeries is Series plus time information. Currently the time information is not as well designed due to complications with timezones and other aspects of datetime. So the recommendation is to keep the time information as string, work with Series, and save timeseries with string column of time.

Series is accessed with `$` while timeseries is accessed with `$$` after variable type.

Series vs Attributes

While you can assign an array to a series variable, it will internally convert that to the series. And similarly assigning series to attribute will convert it into an attribute array.

```
# array
env.x = [1,2,3]
env.x
env$x = [1,2,3]
env$x

env.x + env$x
```

Results:

```
[1, 2, 3]


Series(len: 3, dtype: Integers) [1, 2, 3]

[2, 4, 6]
```

The main thing to keep in mind is that they are stored separately, and the inter-operability is just there to make things easier.

Series with Gaps

Series loaded from CSV/DSS or other sources (each need their own plugins) can have missing data. But because attributes can not be null, only way to make a series with missing data is to generate it with series map.

```
env$x = 1:10
env$x
env$xg = $x -> func(i) {if (i%3 != 0) {return i}}
env$xg
```

Results:

```
Series(len: 10, dtype: Integers) [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]


MaskedSeries(len: 10, dtype: Integers, valid: 7) [1, 2, -, 4, 5, -, 7, 8, -, 10]
```

Data Imputation

You can fill the gaps again using series map. Or of course you can load functions from plugins that do it for you.

Because functions can't take null values, you need to use a default value to check if the series sent any values or not. This might be fixed in the future versions.

```
env $xg -> func(i=false) {if (i==false) {0} else {i}}
```

Results:

```
Series(len: 10, dtype: Integers) [1, 2, 0, 4, 5, 0, 7, 8, 0, 10]
```

You can also use multiple series in series map to impute data based on other series

```
network.load_str("a -> b")
node[a]$x = 1:8
node[b]$x = node[a]$x -> func(i) {if (i%3 != 0) {i * 10}}
nm $x

# filling
node[b]$xf = ($x, input$x) -> func(b=false, a=false) {if (b == false) {a} else {b}}
node[b] $xf
```

Results:

```
{
  b = MaskedSeries(len: 8, dtype: Integers, valid: 6) [10, 20, -, 40, 50, -, 70, 80],
  a = Series(len: 8, dtype: Integers) [1, 2, 3, 4, 5, 6, 7, 8]
}


Series(len: 8, dtype: Integers) [10, 20, 3, 40, 50, 6, 70, 80]
```

Conversion of TimeSeries and Series

If you want to convert series into timeseries, you need another series with same length to be the time component.

```
env$times = ["1920-01-01", "1920-01-02", "1920-01-03", "1920-01-04"]
env$values = [1, 2, 3, 4]

env$$tsvals = timeseries($times, $values)
env$$tsvals
```

Results:

```
TimeSeries([1920-01-01, 1920-01-02, ..., 1920-01-04], values: Series(len: 4, dtype:
Integers) [1, 2, 3, 4])
```

While you can use timeseries directly where you need series, it will only use the series component

```
env$y = env$$tsvals
env$y
```

Results:

```
Series(len: 4, dtype: Integers) [1, 2, 3, 4]
```

If you are doing series map, only one of them needs to be a timeseries, the time information will be carried over to the result.

```
env$$z = (env$$tsvals, env$y) -> func(a, b) {a + b}
env$$z
```

Results:

```
TimeSeries([1920-01-01, 1920-01-02, ..., 1920-01-04], values: Series(len: 4, dtype:
Integers) [2, 4, 6, 8])
```

If you are trying to assign a series values to a timeseries, it will only work if the timeseries already exists.

```
env$$z2 = (env$values, env$y) -> func(a, b) {a + b}
env$$z2
```

Error:

```
TimeSeriesNotFoundError  at Line 1 Column 1: z2
```

If the timeseries already exists, then it will overwrite its values.

```
env$$z = (env$values, env$y) -> func(a, b) {a * b}
env$$z
```

Results:

```
TimeSeries([1920-01-01, 1920-01-02, ..., 1920-01-04], values: Series(len: 4, dtype:
Integers) [1, 4, 9, 16])
```